

Data-Mart-Erstellung in SQL

ERARBEITUNGS- / REFLEXIONSPHASE

Studiengang: Software Development Expert

Genre

Erstellung der Nachschlagetabelle *genre* und Zuordnungstabelle *book_genre*

```
CREATE TABLE genre
(
    genre_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name      varchar(50) NOT NULL,

    CONSTRAINT uc_genre UNIQUE (name)
);
```

```
CREATE TABLE book_genre
(
    book_id  bigint,
    genre_id bigint,

    CONSTRAINT pk_book_genre PRIMARY KEY (book_id, genre_id),
    CONSTRAINT fk_book_genre_genre FOREIGN KEY (genre_id) REFERENCES genre (genre_id),
    CONSTRAINT fk_book_genre_book FOREIGN KEY (book_id) REFERENCES book (book_id)
    ON DELETE CASCADE
);
```

Da Bücher mehreren Genres zugeordnet sein können, wird zunächst eine Nachschlagetabelle mit der Bezeichnung (name) des Genres und die eindeutige technische ID (genre_id).

Die Bezeichnung des Genres ist erforderlich und wird daher als NOT NULL angelegt. Zudem sollten die Werte in einer Nachschlagetabelle eindeutig sein, um bei Zuordnungen konsistente Ergebnisse sicherzustellen. Daher wird das Attribut name mit einem UNIQUE-Constraint versehen.

Die Zuordnungstabelle *book_genre* wird ausschließlich mit den Fremdschlüsselbeziehungen zu Buch (book) und Genre (genre) modelliert. Beide Attribute bilden zusammen den Primärschlüssel, da die Kombination aus Buch und Genre eindeutig sein muss.

Da in diesem Fall jedoch eine Zuordnung von Genre zu Buch nicht mehr sinnvoll ist, wenn ein Buch gelöscht wird, wird das Datenbankmanagementsystem über die referenzielle Aktion *ON DELETE CASCADE* angewiesen, beim Löschen des verknüpften Buches auch den Eintrag in der Zuordnungstabelle zu löschen.

Genre

Testprozedur und Ergebnis

```
CREATE OR REPLACE PROCEDURE getOrCreateGenre(p_name varchar(50),
                                             OUT p_id BIGINT)
    LANGUAGE plpgsql AS
$$
BEGIN
    INSERT INTO genre (name) VALUES ( name p_name)
    RETURNING genre_id INTO p_id;
    RAISE NOTICE 'New genre successfully created. ID: %', p_id;

EXCEPTION
    WHEN unique_violation THEN
        SELECT genre_id INTO p_id from genre g WHERE g.name = p_name;
        RAISE NOTICE 'Genre already exists. Return ID %', p_id;
END;
```

```
[2026-08-26 09:30:53] iuLendMyBook.public> CALL getOrCreateGenre('Comedy', NULL)
New genre successfully created. ID: 16
[2026-08-26 09:30:53] 1 row retrieved starting from 1 in 327 ms (execution: 10 ms, fetching: 317 ms)
[2026-08-26 09:30:53] iuLendMyBook.public> CALL getOrCreateGenre('Thriller', NULL)
Genre already exists. Return ID 3
[2026-08-26 09:30:54] 1 row retrieved starting from 1 in 548 ms (execution: 6 ms, fetching: 542 ms)
[2026-08-26 09:30:54] iuLendMyBook.public> CALL getOrCreateGenre(NULL, NULL)
[2026-08-26 09:30:54] [23502] ERROR: null value in column "name" of relation "genre" violates
not-null constraint
[2026-08-26 09:30:54] Detail: Failing row contains (18, null).
```

Mit Hilfe der Prozedur wird ein Eintrag in der Tabelle *genre* erzeugt.

Ist ein Genre bereits vorhanden, wird die ID des bestehenden Eintrags zurückgegeben. Andernfalls wird ein neuer Eintrag angelegt und die ID dieses Eintrags als Ergebnis geliefert.

Die Testfälle zeigen, dass über die Prozedur erfolgreich ein neues Genre angelegt werden kann. Ist dieses Bereits vorhanden wird eine Meldung und die Genre-ID des vorhandenen Eintrags zurückgegeben.

Versucht man ein Genre ohne Bezeichnung anzulegen, wird eine NOT-NULL-Constraint violation ausgelöst.

Sprache

Erstellung der Nachschlagetabelle *language*

```
CREATE TABLE language
(
    language_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name          varchar(50) NOT NULL,
    iso_code      varchar(5)  NOT NULL,

    CONSTRAINT uc_lang UNIQUE (name),
    CONSTRAINT uc_lang_iso UNIQUE (iso_code),
    CONSTRAINT chk_lang_isoCode CHECK (iso_code ~ '^[a-z]{2}(-[A-Z]{2})?$',)
);
```

Für eine zielgerichtete Suche wird eine weitere Nachschlagetabelle erstellt, die die Buchsprachen repräsentiert. Neben der technischen ID und der Sprachbezeichnung, wird zudem der ISO-Code der Sprache erfasst.

Für sinnvolle Verwendung der Sprache sollten die Felder *name* und *iso-code* immer gefüllt sein (NOT NULL Constraint). Um Sprachvarianten unterscheiden zu können, aber dennoch Doppelungen abzusichern werden beide Felder mit einem UNIQUE-Constraint belegt. Varianten können somit über einen Zusatz im Feld name, z.B. Englisch (USA) - en-US oder Englisch (UK) - en-GB, erstellt werden.

Zudem ist die Semantik der ISO-Codes eingrenzbar, sodass mit einem Check eingegebene Werte direkt überprüft werden können. Der reguläre Ausdruck lässt Codes mit 2 Kleinbuchstaben oder 2 Klein- und 2 Großbuchstaben (getrennt durch -) zu, z.B. en-US

Sprache

Testprozedur und Ergebnis

```
CREATE OR REPLACE PROCEDURE getOrCreateLanguage(p_name varchar(50), p_isoCode varchar(5),
                                                OUT p_id BIGINT)
    LANGUAGE plpgsql AS
$$
BEGIN
    INSERT INTO language (name, iso_code)
    VALUES ( name p_name, iso_code p_isoCode)
    RETURNING language_id INTO p_id;
    RAISE NOTICE 'New language successfully created. ID: %', p_id;

EXCEPTION
    WHEN unique_violation THEN
        SELECT language_id INTO p_id from language l WHERE l.name = p_name or
                                                    l.iso_code = p_isoCode;
        RAISE NOTICE 'Language already exists. Return ID: %', p_id;
END;
$;
```

```
[2026-08-26 09:31:47] iuLendMyBook.public> CALL getOrCreateLanguage('German (Austria)', 'de-AT',
NULL)
New language successfully created. ID: 20
[2026-08-26 09:31:48] 1 row retrieved starting from 1 in 357 ms (execution: 11 ms, fetching: 346 ms)
[2026-08-26 09:31:48] iuLendMyBook.public> CALL getOrCreateLanguage('Slovak', 'sk_SK', NULL)
[2026-08-26 09:31:48] [23514] ERROR: new row for relation "language" violates check constraint
"chk_lang_isocode"
[2026-08-26 09:31:48] Detail: Failing row contains (21, Slovak, sk_SK).
```

Die Prozedur ist analog zur Prozedur zum Test der Genres aufgebaut und gibt im Fall einer UNIQUE-Constraint Verletzung die ID des bereits vorhanden Datensatzes zurück.

Die Testfälle zeigen, dass neue Einträge erfolgreich angelegt werden können, es sei denn das CHECK-Constraint wird verletzt wie im zweiten Testfall: Das Einfügen der Sprache *Slovak* schlägt fehl, da der String des ISO-Codes ein nicht zulässiges Trennzeichen enthält.

Land und Ort

Erstellung der Nachschlagetabellen *country* und *location*

```
CREATE TABLE country
(
    country_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name        varchar(50) NOT NULL,
    iso_code    varchar(2) NOT NULL,

    CONSTRAINT uc_country_name UNIQUE (name),
    CONSTRAINT uc_country_iso  UNIQUE (iso_code),
    CONSTRAINT chk_country_isoCode CHECK (iso_code ~ '^[A-Z]{2}$')
);
```

```
CREATE TABLE location
(
    location_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    postal_code varchar(10) NOT NULL,
    city        varchar(50) NOT NULL,
    country_id  bigint NOT NULL,

    CONSTRAINT uc_location_name UNIQUE (postal_code, city, country_id),
    CONSTRAINT fk_location_country FOREIGN KEY (country_id) REFERENCES country (country_id)
);
```

Zur vollständigen Angabe von Adressen sind Orte und Länderinformationen anzugeben. Da Orte und Länderangaben eindeutig sind, bietet sich im Sinne einer korrekten Normalisierung die Anlage eigener Entitäten an. Diese werden als Nachschlagetabellen ergänzt, sodass aus einem Adressdatensatz darauf verwiesen werden kann.

Zur eindeutigen Identifikation der Länder, werden diese mit Bezeichnung (name) und dem ISO-Code (iso_code) angelegt. Beide Angaben werden mit dem NOT NULL-Constraint als verpflichtend gekennzeichnet. Zudem wird die durch das CHECK-Constraint vorgegeben, dass ISO-Codes nur im korrekten Format angegeben werden können, z.B. US.

Orte hingegen lassen sich über die Kombination aus Postleitzahl (postal_code), Name (city) und Land (country) eindeutig identifizieren. Daher wird ein zusammengesetztes UNIQUE-Constraint für alle drei Attribute definiert.

Land und Ort

Testprozedur und Ergebnis

```
CREATE OR REPLACE PROCEDURE getOrCreateLocation(p_postal_code varchar(10), p_city varchar(50),
                                                p_country varchar(50),
                                                OUT p_location_id BIGINT)

LANGUAGE plpgsql AS
$$
DECLARE
    v_country_id BIGINT;
    v_notice      text;
BEGIN
    SELECT country_id INTO v_country_id from country where name = p_country;

    IF v_country_id IS NULL THEN
        RAISE EXCEPTION 'Country "%" could not be found. Please insert country first', p_country;
    ELSE
        SELECT location_id INTO p_location_id FROM location l
        where l.postal_code = p_postal_code and l.city = p_city and l.country_id = v_country_id;
    END IF;
END;
```

```
[2026-08-26 09:57:48] iuLendMyBook.public> CALL getOrCreateCountry('Chile', 'CL', NULL)
New country successfully created. ID: 30
[2026-08-26 09:57:48] iuLendMyBook.public> CALL getOrCreateCountry('Suisse', 'SUI', NULL)
[2026-08-26 09:57:48] [22001] ERROR: value too long for type character varying(2)
[2026-08-26 09:58:27] iuLendMyBook.public> CALL getOrCreateCountry('Morocco', 'M9', NULL)
[2026-08-26 09:58:27] [23514] ERROR: new row for relation "country" violates check constraint
"chk_country_isocode"
[2026-08-26 09:58:27] Detail: Failing row contains (31, Morocco, M9).
[2026-08-26 09:59:35] iuLendMyBook.public> CALL getOrCreateLocation('34225', 'Baunatal', 'Germany', NULL)
New location with ID 41 successfully created.
[2026-08-26 09:59:35] iuLendMyBook.public> CALL getOrCreateLocation('11458', 'Cardiff', 'Wales', NULL)
[2026-08-26 09:59:35] [P0001] ERROR: Country "Wales" could not be found. Please insert country first
```

Während die Prozedur zur Anlage eines Landes dem bereits bekannten Schema folgt, ist der Aufbau für den Ort umfassender. Es wird zunächst die ID des Landes aus der Nachschlagetabelle des Landes ermittelt. Ist das Land nicht vorhanden, wird eine Ausnahme zurückgegeben. Im Anschluss folgt Anhand der Attribute des UNIQUE Constraints die Prüfung, ob ein Eintrag bereits besteht. In diesem Fall wird die ID zurückgegeben. Andernfalls wird ein neuer Eintrag angelegt.

Die Testfälle zeigen, dass ein falscher ISO-Code (dreistellig bzw. alphanummerisch) aufgrund der Feldlängenbegrenzung und des CHECK-Constraints abgelehnt werden.

Das Einfügen eines neuen Ortes hingegen ist erfolgreich, solange das Land vorhanden ist. Der nicht vorhandene Eintrag „Wales“ führt dagegen zu einem Fehler bei Ausführung der Prozedur.

Adresse

Erstellung der Stammdatentabelle *address*

```
CREATE TABLE address
(
  address_id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  street      varchar(255) NOT NULL,
  number      varchar(10),
  location_id bigint      NOT NULL,
  longitude   numeric(9, 6),
  latitude    numeric(9, 6),

  CONSTRAINT fk_address_location FOREIGN KEY (location_id) REFERENCES location (location_id),
  CONSTRAINT chk_address_longitude CHECK (longitude BETWEEN -180 AND 180),
  CONSTRAINT chk_address_latitude CHECK (latitude BETWEEN -90 AND 90)
);
```

Die Adresse wird im Rahmen der Buchtausch-App gleichsam für Benutzer und Verlage verwendet. Sie setzt sich aus der Straße und Hausnummer sowie über eine Fremdschlüsselbeziehung zur Tabelle Ort (location) zusammen. Die Attribute Längen- (longitude) und Breitengrad (latitude) sind zusätzliche Attribute, um bei für die Anzeige der Suchergebnisse auch eine lokale/regionale Suche zu unterstützen.

Um einen Versand oder eine Abholung durchführen zu können, sind die Informationen zu Straße und Ort als erforderliche Attribute gekennzeichnet, während die Hausnummer auch NULL-Werte annehmen darf.

Die Hausnummer wird als String angelegt, damit auch alphanummerische Eingaben wie 12a, möglich sind. Längen- und Breitengrad haben eine feste Syntax. Da beide nur Werte in einem definierten Intervall (Breitengrad: -90 bis +90, Längengrad: -180 bis +180) annehmen können, werden zusätzlich Prüfkriterien gesetzt.

Adresse

Testprozedur und Ergebnis

```
CREATE OR REPLACE PROCEDURE getOrCreateAddress(p_street varchar(255), p_number varchar(10), p_city varchar(100),
                                                p_postal_code varchar(10), p_country varchar(50),
                                                p_longitude numeric(9, 6), p_latitude numeric(9, 6),
                                                OUT p_address_id BIGINT)

LANGUAGE plpgsql AS
$$
DECLARE
    v_location_id BIGINT;
BEGIN
    CALL getOrCreateLocation( p_postal_code p_postal_code, p_city p_city, p_country p_country, v_location_id);

    INSERT INTO address (street, number, location_id, longitude, latitude)
    VALUES ( street p_street, number p_number, location_id v_location_id, longitude p_longitude,
            latitude p_latitude)

END;
$$;
```

```
[2026-08-26 10:26:49] iuLendMyBook> CALL getOrCreateAddress('Musterstraße', '14a', 'Berlin', '10178'
, 'Germany', 13.41321, 52.5219, NULL )

Found location with ID: 16
New address successfully created. ID: 41
[2026-08-26 10:26:49] iuLendMyBook> CALL getOrCreateAddress('Kastanienallee', '12', 'Berlin', '10178'
, 'Germany', 92.41321, -195.5219, NULL )

[2026-08-26 10:26:49] [23514] ERROR: new row for relation "address" violates check constraint "chk_address_latitude"
[2026-08-26 10:26:49] Detail: Failing row contains (42, Kastanienallee, 12, 16, 92.413210, -195.521900).
[2026-08-26 10:28:21] iuLendMyBook.public> CALL getOrCreateAddress('Lombard Street', '1', 'San Francisco', 'CA 94111'
, 'United States', -122.40277, 37.80413, NULL )

New location with ID 44 successfully created.
New address successfully created. ID: 45
```

Innerhalb der Prozedur wird zunächst über die bereits vorgestellt Prozedur *getOrCreateLocation* die ID des Ortes ermittelt und dabei bei Bedarf ein neuer Ort angelegt. Ist die Adresse bereits vorhanden, wird die ID der Adresse zurückgegeben, anderenfalls wird die Adresse in der Tabelle neu hinzugefügt.

Wie in den Testfällen erkennbar ist, wird zunächst ein Ort gesucht bzw. neu angelegt bevor eine neue Adresse erstellt wird.

Zudem wird ersichtlich, dass bei fehlerhafter Angabe von Längen- und Breiten das definierte CHECK-Constraint das Einfügen des Eintrags verhindert.

Verlag

Erstellung der Stammdatentabelle *publisher*

```
CREATE TABLE publisher
(
    publisher_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name         varchar(255) NOT NULL,
    website      varchar(255),
    address_id   bigint,

    CONSTRAINT uk_publisher UNIQUE NULLS NOT DISTINCT (name, address_id),
    CONSTRAINT fk_publisher_address FOREIGN KEY (address_id) REFERENCES address (address_id)
);
```

Die meisten Bücher werden durch Verlage veröffentlicht. Da ein Verlag in der Regel mehrere Bücher verlegt, bietet sich eine eigene Tabelle an, um über einen Fremdschlüssel von einem Buch darauf zu verweisen.

Um den Verlag identifizieren zu können, ist die Bezeichnung des Verlags (name) als NOT NULL gekennzeichnet, während der Fremdschlüssel auf eine Adresse und die Webseite des Verlages optionale Informationen sind.

Da jedoch der Name eines Verlages nicht eindeutig ist, aber davon auszugehen ist, dass die Kombination aus Verlag und Adresse eindeutig ist, wird als UNIQUE-Constraint die Kombination beider Felder festgelegt. Im Vergleich zu bereits angelegten Tabellen ist hier jedoch der Zusatz *NULLS NOT DISTINCT*. Dieser ist notwendig, da die Adresse leer sein kann, aber Postgres NULL-Werte in UNIQUE-Constraints im Standard nicht als gleich betrachtet. Der Zusatz stellt sicher, dass bei gleichem Namen, aber fehlender Adresse kein neuer Eintrag angelegt wird.

Verlag

Testprozedur und Ergebnis

```
CREATE OR REPLACE PROCEDURE getOrCreatePublisher(p_name varchar(255), p_website varchar(255), p_street varchar(255),
                                                p_number varchar(10), p_city varchar(50), p_postal_code varchar(10),
                                                p_country varchar(50),
                                                OUT v_publisher_id BIGINT)

LANGUAGE plpgsql AS
$$
DECLARE
    v_address_id BIGINT;
BEGIN
    IF p_city IS NOT NULL THEN
        CALL getOrCreateAddress(p_street, p_number, p_city, p_postal_code, p_country, NULL, NULL, v_address_id);
    END IF;

    IF v_address_id IS NULL THEN
        SELECT publisher_id INTO v_publisher_id FROM publisher p WHERE p.name = p_name and p.address_id IS NULL;
    ELSE
        SELECT publisher_id INTO v_publisher_id FROM publisher p WHERE p.name = p_name and p.address_id = v_address_id;
    END IF;

    IF v_publisher_id IS NULL THEN
        INSERT INTO publisher (name, website, address_id)

```

```
[2026-08-26 10:46:41] iuLendMyBook.public> CALL getOrCreatePublisher('HarperCollins Publishers Ltd', 'https://harcollins.co.uk', 'Robroyston Gate',
                                                                '1', 'Glasgow', 'G33 1JN', 'United Kingdom', NULL)

New location with ID 45 successfully created.
New address successfully created. ID: 46
New publisher successfully created. ID: 21
[2026-08-26 10:46:41] iuLendMyBook.public> CALL getOrCreatePublisher('Das Verlagshaus', 'http://dasverlagshaus.de', NULL,
                                                                NULL, NULL, NULL, NULL, NULL)

New publisher successfully created. ID: 22
[2026-08-26 10:46:42] iuLendMyBook.public> INSERT INTO publisher (name, website, address_id) VALUES ('Das Verlagshaus', 'http://dasverlagshaus', NULL)
[2026-08-26 10:46:42] [23505] ERROR: duplicate key value violates unique constraint "uk_publisher"
[2026-08-26 10:46:42] Detail: Key (name, address_id)=(Das Verlagshaus, null) already exists.
```

Die Prozedur erstellt einen neuen Eintrag in der Tabelle Verlag, sofern dieser nicht bereits existiert. Durch den Aufruf *getOrCreateAddress* wird zudem sichergestellt, dass die übergebene Adresse mit dem Eintrag verknüpft wird.

Die Testfälle zeigen, dass die Anlage eines neuen Verlags mit einer nicht vorhandenen Adresse zunächst dafür sorgt, dass ein Ort, danach eine Adresse und zuletzt der Verlag angelegt werden.

Auch die Anlage eines Verlages ohne Adresse ist erfolgreich.

Im letzten Testfall wird versucht den Verlag ohne Adresse erneut hinzuzufügen. Aufgrund der *NULLS NOT DISTINCT* Klausel wird das Einfügen verhindert.

Buch

Erstellung der Stammdatentabelle *book*

```
CREATE TABLE book
(
  book_id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  title            varchar(255) NOT NULL,
  isbn             varchar(13)  NOT NULL,
  description       text,
  publication_year  smallint,
  edition          smallint,
  language_id      bigint,
  publisher_id     bigint,

  CONSTRAINT fk_book_language FOREIGN KEY (language_id) REFERENCES language (language_id)
    ON DELETE SET NULL,
  CONSTRAINT fk_book_publisher FOREIGN KEY (publisher_id) REFERENCES publisher (publisher_id)
    ON DELETE SET NULL,
  CONSTRAINT uc_book_isbn UNIQUE (isbn),
  CONSTRAINT chk_book_isbn CHECK ( isbn ~ '^([0-9]){13}$' ),
  CONSTRAINT chk_book_publication_year CHECK ( publication_year BETWEEN 1000 AND 3000 ),
  CONSTRAINT chk_book_edition CHECK ( edition > 0 )
);
```

Die Entität *Buch* (*book*) stellt das zentrale Element innerhalb der Buchtausch-App dar. Der Titel (*title*) und die ISBN (*isbn*) dienen dabei der Identifikation des Buches, sodass sie als Pflichtfelder gekennzeichnet werden. Da die ISBN international eindeutig ist, wird diese zudem mit einem UNIQUE-Constraint versehen.

Die referenziellen Aktionen für die Fremdschlüsselbeziehungen zu Sprache und Verlag werden auf *ON DELETE SET NULL* gesetzt, sodass ein Löschen von Einträgen möglich ist, auch wenn diese von einem Buch referenziert werden. Der Wert am Buch wird dabei auf NULL gesetzt.

Weiterhin werden CHECK-Constraints definiert, die sicherstellen, dass nur syntaktisch korrekte ISBN mit einer Länge von 13 numerischen Zeichen eingetragen werden können und die Felder Ausgabe (*edition*) und Erscheinungsjahr (*publication_year*) sinnvolle Werte enthalten.

Buch

Testprozedur und Ergebnis

```
CREATE OR REPLACE PROCEDURE createOrUpdateBook(p_title varchar(255), p_isbn varchar(13), p_description text,  
p_year smallint, p_edition smallint, p_language_iso_code varchar(5),  
p_publisher_name varchar(255), p_author_array varchar(110)[],  
p_genre_array varchar(50)[], OUT n_book_id BIGINT)
```

```
BEGIN
```

```
CREATE OR REPLACE PROCEDURE deleteBook(p_isbn varchar(13), p_current_userid BIGINT)
```

```
LANGUAGE plpgsql AS
```

```
$$
```

```
BEGIN
```

```
IF NOT EXISTS (SELECT 1  
FROM user_account ua  
INNER JOIN user_role ur 1<->1..n: on ua.user_id = ur.user_id  
INNER JOIN role r 1..n<->1: on ur.role_id = r.role_id  
WHERE r.name = 'ADMIN'  
and ua.user_id = p_current_userid) THEN  
RAISE EXCEPTION 'Only administrators are allowed to delete book titles';  
END IF;
```

```
[2026-08-26 12:52:50] iuLendMyBook.public> CALL createOrUpdateBook('Lord of the Rings - Fellowship of the Ring', '9780261102354',  
The first part of The Lord of the Rings. Frodo Baggins inherits the power for the final  
journey to destroy the One Ring.  
ARRAY ['J.R.R. Tolkien'], ARRAY ['Science-Fiction'], NULL)  
Book "Lord of the Rings - Fellowship of the Ring" with ID 27 successfully created  
Genre already exists. Return ID 13  
New author successfully created. ID: 46  
[2026-08-26 12:52:51] iuLendMyBook.public> CALL createOrUpdateBook('Lord of the Rings - Fellowship of the Ring', '9780261102354',  
The first part of The Lord of the Rings. Frodo Baggins inherits the power for the final  
journey to destroy the One Ring.  
ARRAY ['J.R.R. Tolkien'], ARRAY ['Adventure', 'Science-Fiction'], NULL)  
Book "Lord of the Rings - Fellowship of the Ring" with ID 27 successfully updated  
New genre successfully created. ID: 36  
Genre already exists. Return ID 13  
Found author with ID 46  
[2026-08-26 12:52:51] iuLendMyBook.public> CALL deleteBook('9780261102354', 3)  
[2026-08-26 12:52:51] [P0001] ERROR: Only administrators are allowed to delete book titles  
[2026-08-26 12:52:58] iuLendMyBook.public> CALL deleteBook('9780261102354', 1)  
Book with ISBN 9780261102354 successfully deleted  
[2026-08-26 12:52:58] iuLendMyBook.public> CALL deleteBook('9780261102354', 1)  
[2026-08-26 12:52:58] [P0001] ERROR: Book with ISBN 9780261102354 cannot be deleted as it does not exist
```

Die Testprozedur *createOrUpdateBook* ist stark verkürzt dargestellt. Wie bei den bereits vorgestellten Prozeduren wird anhand der Parameter ermittelt, ob ein Buch bereits vorhanden ist. Jedoch werden abweichend dazu beim Auffinden eines Datensatzes die übergebenen Parameter zum Aktualisieren der Informationen genutzt.

Um dabei sicherzustellen, dass Zuordnungen von Autoren und Genres konsistent bleiben, werden die Zuordnungen beim Auffinden eines Buches zunächst entfernt. Am Ende der Prozedur werden die Zuordnungen anhand der übergebenen Parameter jedoch wieder hinzugefügt .

Mit der Prozedur *deleteBook* (verkürzt) wird zudem die Vorgabe abgebildet, dass Bücher nur mit der Rolle *ADMIN* gelöscht werden können. Die Rolle wird dabei anhand des übergebenen Users geprüft.

Die Prozeduren zum Löschen, zeigen, dass das Löschen eines Buches durch einen einfachen Nutzer nicht möglich ist, aber mit einem Administrator erfolgreich abgeschlossen werden kann.

Autor

Erstellung der Stammdatentabelle *author* und der Zuordnungstabelle *book_author*

```
CREATE TABLE author
(
    author_id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    academic_title varchar(10),
    first_name     varchar(50),
    last_name      varchar(50) NOT NULL,

    CONSTRAINT uk_author UNIQUE NULLS NOT DISTINCT (academic_title, first_name, last_name)
);
```

```
CREATE TABLE book_author
(
    book_id  bigint,
    author_id bigint,

    CONSTRAINT pk_book_author PRIMARY KEY (book_id, author_id),
    CONSTRAINT fk_book_author_author FOREIGN KEY (author_id) REFERENCES author (author_id),
    CONSTRAINT fk_book_author_book FOREIGN KEY (book_id) REFERENCES book (book_id)
    ON DELETE CASCADE
);
```

Aufgrund der m:n Beziehung von Buchtitel und Autor bietet es sich an, die Autoren als eigene Entität anzulegen und die Zuordnung von Büchern und Autoren in einer Zuordnungstabelle vorzunehmen.

Ein Autor hat dabei die Merkmale *Titel*, *Vorname* und *Nachname*. Entscheidend bei der Angabe im Kontext von Büchern ist der Nachname (*last_name*), sodass dieser als Pflichtangabe (NOT NULL) gekennzeichnet wird. Es wird davon ausgegangen, dass ein Name in der Kombination aus Titel, Vor- und Nachname eindeutig ist, sodass über diese Attribute das UNIQUE-Constraint festgelegt wird.

Die Zuordnungstabelle *book_author* wird ausschließlich mit den Fremdschlüsselbeziehungen zu Buch (*book*) und Autor (*author*) modelliert. Beide Attribute bilden zusammen den Primärschlüssel, da die Kombination aus Buch und Autor eindeutig sein muss.

Wird ein Buch gelöscht, ist die Zuordnung zum Autor nicht mehr sinnvoll ist. Die referenzielle Aktion *ON DELETE CASCADE* sorgt dafür, dass beim Löschen des verknüpften Buches auch der Eintrag in der Zuordnungstabelle gelöscht wird.

Autor

Testprozedur und Ergebnis

```
CREATE OR REPLACE PROCEDURE getOrCreateAuthor(p_firstname varchar(50), p_lastname varchar(50),  
                                                p_academic_title varchar(10),  
                                                OUT v_author_id BIGINT)  
LANGUAGE plpgsql AS
```

```
..NLB,  
$$;
```

```
SELECT isbn, title, string_agg(concat(a.last_name, ', ', a.first_name), '; ') authors from author a  
INNER JOIN public.book_author ba 1<->1..n: on a.author_id = ba.author_id  
INNER JOIN public.book b 1..n<->1: on ba.book_id = b.book_id  
GROUP BY title, isbn LIMIT 10;
```

	isbn	title	authors
1	9781000002003	The Glass Horizon	Brighton, Marcus; van den Hale, Elena
2	9781000002010	Beneath the Quiet Bridge	de Carter, Iris
3	9781000002027	The Orchard of Borrowed Names	Willowmere, Noah; von Mercer, Amelia
4	9781000002034	Practical Systems Thinking	Stonebridge, Gavin
5	9781000002041	The Crown of Ember Bay	Ashwell, Lydia
6	9783000001000	Das Echo der stillen Stadt	Kernwald, Jonas; von Falkenried, Mara
7	9783000001017	Jenseits des kupfernen Waldes	zu Morgenfeld, Leonie
8	9783000001024	Logik im Alltag	van Dorn, Tobias
9	9783000001031	Die letzte Fahrt nach Nordhafen	ten Broek, Klara; Wellen, Henrik
10	9783000001048	Sommer unter fremden Sternen	de Winter, Sina

Die Anlage eines Autors erfolgt über die Prozedur *getOrCreateAuthor*. Diese folgt dem Aufbau der Prozeduren zur Anlage von Einträgen in Nachschlagetabellen.

Mit dem Aufruf der Buchtitel mit den zugewiesenen Autoren wird gezeigt, dass ein Buch einen oder mehrere Autoren besitzen kann.

Für eine einfachere Darstellung wurde dabei die Aggregatfunktion *string_agg* genutzt, die je Buch die Ergebnisse der Autoren zusammenfasst und diese konkateniert und dem gruppierten Eintrag hinzufügt.

Benutzer

Erstellung der Stammdatentabelle *user_account*

```
CREATE TABLE user_account
(
    user_id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    academic_title   varchar(10),
    first_name       varchar(50),
    last_name        varchar(50) NOT NULL,
    email            varchar(255) NOT NULL,
    phone            varchar(20),
    status_id        bigint      NOT NULL DEFAULT 1,

    CONSTRAINT uc_user_email UNIQUE (email),
    CONSTRAINT fk_user_status FOREIGN KEY (status_id) REFERENCES status (status_id),
    CONSTRAINT chk_user_phone CHECK (phone ~ '^[+][0-9 -]+$'),
    CONSTRAINT chk_user_email CHECK (email ~ '^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[A-Za-z]{2,}$')
);
```

```
CREATE TABLE status
(
    status_id        bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name             varchar(10) NOT NULL,

    CONSTRAINT uc_status_name UNIQUE (name)
);
```

Neben den Büchern stellt die Entität Benutzer (*user_account*) ein zentrales Element der Buchtausch-App dar. In der Tabelle werden Nachname und E-Mailadresse als Pflichtfelder (NOT NULL) erfasst, da sie für die Identifikation und die Kommunikation benötigt werden. Der Vorname hingegen wird als optional definiert. So ist es auch möglich z.B. juristische Personen als Nutzer anzugeben.

Zudem wird ein Benutzerstatus angelegt. Dieser ermöglicht es im Betrieb der App Nutzer zu sperren, ohne sie und ihre Historie zu löschen. Dazu wird eine Nachschlagetabelle erstellt, die die Status mit ID und Bezeichnung aufnimmt, wobei der UNIQUE-Key auf der Statusbezeichnung liegt. Da bei Anlage eines Benutzers davon ausgegangen wird, dass er aktiv ist, wird die *status_id* im Standard auf 1 gesetzt.

Für Telefonnummer und E-Mailadresse werden weitere Prüfkriterien hinterlegt., sodass sichergestellt wird, dass sie syntaktisch korrekt sind.

Zudem wird für die E-Mailadresse ein UNIQUE-Constraint ergänzt, da sie für Kommunikation verwendet wird und eindeutig sein muss.

Benutzer

Testprozedur und Ergebnis

```
CREATE OR REPLACE PROCEDURE getOrCreateUserAccount(p_fullname varchar(110), p_email varchar(255), p_phone varchar(20),
                                                    OUT p_user_id BIGINT)
LANGUAGE plpgsql AS
$$
CREATE OR REPLACE PROCEDURE updateUserAccount(p_fullname varchar(110), p_email varchar(255), p_newEmail varchar(255),
                                              p_phone varchar(20), p_status varchar(10),
                                              OUT p_user_id BIGINT)
LANGUAGE plpgsql AS
$$
DECLARE
    v_status_id BIGINT;
BEGIN
    CALL getOrCreateUserAccount(p_fullname, p_email, p_phone, p_user_id);
    SELECT status_id INTO v_status_id FROM status WHERE name = p_status;

    UPDATE user_account
    SET first_name  = CASE WHEN p_fullname IS NOT NULL THEN getFirstName( p_complete_name p_fullname) ELSE first_name END,
        last_name   = CASE WHEN p_fullname IS NOT NULL THEN getLastName( p_complete_name p_fullname) ELSE last_name END,
        academic_title = CASE WHEN p_fullname IS NOT NULL THEN getTitle( p_complete_name p_fullname) ELSE academic_title END,
        email       = CASE WHEN p_newEmail IS NULL THEN email ELSE p_newEmail END,
        phone       = CASE WHEN p_phone IS NULL THEN phone ELSE p_phone END,
        status_id   = CASE WHEN v_status_id IS NULL THEN status_id ELSE v_status_id END
    WHERE user_id = p_user_id;
    RAISE NOTICE 'User with ID % updated successfully.', p_user_id;
END;
$$;
```

```
[2026-08-26 14:28:49] iuLendMyBook.public> CALL getOrCreateUserAccount('Dr. Max Meier', 'max.meier@abc.de', '+49 1515 668899', NULL)
New user successfully created. ID: 31
[2026-08-26 14:28:49] iuLendMyBook.public> CALL updateUserAccount('Dr. Max Müller', 'max.meier@abc.de', 'max.mueller@web.de', NULL, NULL, NULL)
User with ID 31 updated successfully.
[2026-08-26 14:28:50] iuLendMyBook.public> CALL updateUserAccount('Christian Steffen', 'c.steffen@gmail.com', NULL, NULL, 'BLOCKED', NULL)
New user successfully created. ID: 32
User with ID 32 updated successfully.
[2026-08-26 14:28:52] iuLendMyBook.public> CALL getOrCreateUserAccount('Karla Karstens', 'k.karstens@#23w.de', NULL, NULL)
[2026-08-26 14:28:52] [23514] ERROR: new row for relation "user_account" violates check constraint "chk_user_email"
[2026-08-26 14:28:52] Detail: Failing row contains (33, null, Karla, Karstens, k.karstens@#23w.de, null, ACTIVE).
[2026-08-26 14:28:55] iuLendMyBook.public> CALL updateUserAccount(NULL, 'c.steffen@gmail.com', NULL, NULL, 'SUSPENDED', NULL)
[2026-08-26 14:28:55] [23514] ERROR: new row for relation "user_account" violates check constraint "chk_user_status"
[2026-08-26 14:28:55] Detail: Failing row contains (32, null, Christian, Steffen, c.steffen@gmail.com, null, SUSPENDED).
```

Ähnlich wie die vorgestellten Prozeduren gibt *getOrCreateUserAccount* einen Benutzer zurück oder legt diesen an. Dabei wird der Benutzer anhand des eindeutigen Schlüssels *email* ermittelt.

Die Prozedur *updateUserAccount* hingegen aktualisiert einen Benutzer anhand der übergebenen Parameter. Durch Aufruf der Prozedur *getOrCreateUserAccount* wird sichergestellt, dass ein nicht vorhandener Benutzer neu angelegt wird.

Die Testfälle zeigen, dass ein Nutzer zunächst erfolgreich angelegt und im Anschluss aktualisiert wird. Im dritten Testfall wird *updateUserAccount* aufgerufen, ohne dass der Benutzer besteht. Wie vorgesehen wird der Nutzer daraufhin neu angelegt.

Die letzten beiden Testfälle prüfen die Funktion der Prüfkriterien. So werden Einträge mit falscher Syntax der E-Mailadresse oder falschem Status abgelehnt

Benutzeradresse

Erstellung der Zuordnungsentität *user_address*

```
CREATE TABLE user_address
(
    user_address_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id          bigint NOT NULL,
    address_id       bigint NOT NULL,
    address_type_id  bigint NOT NULL,

    CONSTRAINT fk_user_address_user FOREIGN KEY (user_id) REFERENCES user_account (user_id)
        ON DELETE CASCADE,
    CONSTRAINT fk_user_address_address FOREIGN KEY (address_id) REFERENCES address (address_id),
    CONSTRAINT fk_user_address_type FOREIGN KEY (address_type_id) REFERENCES address_type (address_type_id),
    CONSTRAINT uc_user_address UNIQUE (user_id, address_id, address_type_id)
);
```

```
CREATE TABLE address_type
(
    address_type_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name            varchar(50) NOT NULL,

    CONSTRAINT uc_address_type_name UNIQUE (name)
);
```

Die Benutzeradresse (*user_address*) stellt eine besondere Art von Entität dar: Sie löst wie eine Zuordnungstabelle die m:n-Beziehung zwischen Benutzer (*user_account*) und Adresse (*address*) durch Fremdschlüsselbeziehungen auf. Zudem enthält sie mit dem Adresstyp auch eine fachliche Information, sodass sie Entität betrachtet werden kann. Diese Art wird auch als assoziative Entität bezeichnet.

Durch den Adresstyp ist die Kombination von Adresse und Benutzer nicht mehr eindeutig, sodass der Fremdschlüssel auf den Adresstyp zusätzlich in das UNIQUE-Constraint aufgenommen wird. Da

Da die Zuordnung obsolet ist, sobald ein Benutzer gelöscht wird, wird mit ON DELETE CASCADE sichergestellt, dass verwaiste Einträge beim Löschen des Benutzers mitgelöscht werden.

Der Adresstyp stellt eine einfache Nachschlagetabelle dar, die neben dem künstlichen Primärschlüssel die Bezeichnung enthält. Diese stellt auch den UNIQUE-Constraint dar. Initial werden die beiden Ausprägungen *SHIPPING* und *PICK_UP* angelegt.

Benutzeradresse

Testprozedur und Ergebnis

```
CREATE OR REPLACE PROCEDURE getOrCreateUserAddress(p_user_id BIGINT, p_address_id BIGINT,
                                                    p_address_type varchar(10), OUT p_user_address_id BIGINT)
LANGUAGE plpgsql AS
$$
DECLARE
    v_type_id BIGINT;
BEGIN
    SELECT address_type_id INTO v_type_id FROM address_type WHERE name = p_address_type;
    IF(v_type_id IS NULL) THEN
        RAISE EXCEPTION 'Invalid adresstype. Adresstype must be on of ''SHIPPING'' or ''PICK_UP''';
    END IF;

    INSERT INTO user_address (user_id, address_id, address_type_id)
    VALUES ( user_id p_user_id, address_id p_address_id, address_type_id v_type_id)
    RETURNING user_address_id INTO p_user_address_id;
    RAISE NOTICE 'User address with ID % created successfully.', p_user_address_id;

EXCEPTION WHEN unique_violation THEN
    SELECT user_address_id INTO p_user_address_id FROM user_address
    WHERE address_type_id = v_type_id and address_id = p_address_id and user_id = p_user_id;
    RAISE NOTICE 'User address already exists. Returning ID %', p_user_address_id;
END;
$$;
```

```
[2026-08-26 15:27:13] iuLendMyBook.public> CALL getOrCreateUserAddress(getUserByEmail('c.steffen@gmail.com'),getAddress('Alexanderplatz', '1',
                                                                                                     '10178', 'Berlin', 'DE'),'SHIPPING', NULL)

User address with ID 1 created successfully.
[2026-08-26 15:27:13] iuLendMyBook.public> CALL getOrCreateUserAddress(getUserByEmail('c.steffen@gmail.com'), getAddress('Alexanderplatz', '1',
                                                                                                     '10178', 'Berlin', 'DE'),'SHIPPING', NULL)

User address already exists. Returning ID 56
[2026-08-26 15:27:14] iuLendMyBook.public> CALL getOrCreateUserAddress(getUserByEmail('c.steffen@gmail.com'),getAddress('Alexanderplatz', '1',
                                                                                                     '10178', 'Berlin', 'DE'),'PICK_UP', NULL)

User address with ID 1 created successfully.
[2026-08-26 15:27:15] iuLendMyBook.public> CALL getOrCreateUserAddress(getUserByEmail('c.steffen@gmail.com'),getAddress('Alexanderplatz', '1',
                                                                                                     '10178', 'Berlin', 'DE'),'OTHER', NULL)

[2026-08-26 15:27:15] [P0001] ERROR: Invalid adresstype. Adresstype must be on of 'SHIPPING' or 'PICK_UP'
```

Die Prozedur folgt den bekannten Schemata und legt eine neue Zuordnung an. Wird das UNIQUE-Constraint verletzt, wird der betreffende Eintrag zurückgegeben.

Die Testergebnisse spiegeln die Funktion der Prozedur wider. Während im ersten Testfall der Datensatz erfolgreich angelegt werden konnte wird im zweiten kein neuer Eintrag erstellt, sondern die bestehende ID zurückgegeben. Da im dritten Testfall der Adresstyp abweicht, wird ein neuer Eintrag erzeugt.

Ist der Adresstyp ungültig fängt die Prozedur dies mit einer Ausnahme ab und verhindert die Anlage des Datensatzes.

Rolle

Erstellung der Stammdatentabelle *role* und der Zuordnungstabelle *user_role*

```
CREATE TABLE role
(
    role_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name      varchar(20) NOT NULL,

    CONSTRAINT uk_role_name UNIQUE (name)
);
```

```
CREATE TABLE user_role
(
    role_id bigint NOT NULL,
    user_id  bigint NOT NULL,

    CONSTRAINT pk_user_role PRIMARY KEY (role_id, user_id),
    CONSTRAINT fk_user_role_role FOREIGN KEY (role_id) REFERENCES role (role_id)
        ON DELETE CASCADE,
    CONSTRAINT fk_user_role_user FOREIGN KEY (user_id) REFERENCES user_account (user_id)
        ON DELETE CASCADE
);
```

Die Entität Rolle (*role*) ist eine einfache Tabelle, die neben dem künstlichen Primärschlüssel die Bezeichnung der Rolle aufnimmt. Diese muss eindeutig sein (UNIQUE-Constraint), da sie das einzige fachliche Unterscheidungsmerkmal ist.

Die Zuordnungstabelle *user_role* dient der Zuordnung von Rollen zu Benutzern und löst damit die m:n-Beziehung zwischen beiden Entitäten auf. Entsprechend sind beide Attribute als Fremdschlüsselbeziehungen abgebildet. Der Primärschlüssel setzt sich aus beiden Werten zusammen.

Werden die Rolle oder der Benutzer gelöscht, gibt es keine Notwendigkeit für die Zuordnung mehr. Daher wird für beide Fremdschlüssel ON DELETE CASCADE als referenzielle Aktion festgelegt.

Rolle

Testprozedur und Ergebnis

```
CREATE OR REPLACE FUNCTION deleteRole(p_role varchar(50), p_user_id BIGINT) RETURNS BOOLEAN
LANGUAGE plpgsql AS
$$
BEGIN
    IF (p_role = 'ADMIN') THEN
        RAISE EXCEPTION 'The role ''ADMIN'' is a system role and cannot be deleted';
    END IF;
    IF isAdmin(p_user_id p_user_id) THEN
        DELETE from role where name = p_role;
        RAISE NOTICE 'Role successfully deleted';
        RETURN TRUE;
    ELSE
        RAISE EXCEPTION 'You are not allowed to delete a role';
    END IF;
END;
$$;
```

```
[2026-08-26 15:44:53] iuLendMyBook.public> INSERT INTO role (name) VALUES ('New role')
[2026-08-26 15:44:53] 1 row affected in 10 ms
[2026-08-26 15:44:53] iuLendMyBook.public> SELECT deleterole('New role', getuserbyemail('max.meier@web.de'))
[2026-08-26 15:44:53] [P0001] ERROR: You are not allowed to delete a role
[2026-08-26 15:45:03] iuLendMyBook.public> SELECT deleterole('New role', findadminuser())
Role successfully deleted
[2026-08-26 15:45:03] iuLendMyBook.public> SELECT deleterole('ADMIN', findadminuser())
[2026-08-26 15:45:03] [P0001] ERROR: The role 'ADMIN' is a system role and cannot be deleted
```

Da das Löschen einer Rolle Auswirkungen auf die gesamte Anwendung haben kann, sollte sie nur durch einen Administrator gelöscht werden können. Zudem sollte die Rolle ‚ADMIN‘ selbst nicht gelöscht werden, da man sich so von Systemfunktionen ausschließen könnte. Beides wird in der Prozedur *deleteRole* abgebildet.

Die Testfälle prüfen zunächst das Anlegen der Rolle, sowie das anschließende Löschen der gleichen Rolle erfolgreich ab. Die Rolle ‚ADMIN‘ hingegen kann nicht gelöscht werden.

Der abschließende Testfall zeigt, dass eine zugewiesene Rolle aufgrund der referenziellen Aktion erfolgreich gelöscht werden kann.

Buchexemplar

Erstellung der Stammdatentabelle *book_copy* und der Zuordnungstabelle *book_copy_fulfillment*

```
CREATE TABLE book_copy
(
    book_copy_id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    book_id           bigint NOT NULL,
    owner_id          bigint NOT NULL,
    status_id         bigint NOT NULL DEFAULT 1,
    loan_duration_days smallint NOT NULL,
    condition          varchar(50),

    CONSTRAINT fk_book_copy_book FOREIGN KEY (book_id) REFERENCES book (book_id),
    CONSTRAINT fk_book_copy_status FOREIGN KEY (status_id) REFERENCES status (status_id),
    CONSTRAINT fk_book_copy_owner FOREIGN KEY (owner_id) REFERENCES user_account (user_id)
        ON DELETE CASCADE,
    CONSTRAINT chk_book_copy_duration CHECK (loan_duration_days > 0)
);

CREATE TABLE book_copy_fulfillment
(
    fulfillment_type_id bigint,
    book_copy_id        bigint,

    CONSTRAINT pk_book_copy_fulfillment PRIMARY KEY (fulfillment_type_id, book_copy_id),
    CONSTRAINT fk_book_copy_fulfillment_copy FOREIGN KEY (book_copy_id) REFERENCES book_copy (book_copy_id)
        ON DELETE CASCADE,
    CONSTRAINT book_copy_fulfillment_fulfillment FOREIGN KEY (fulfillment_type_id) REFERENCES
        fulfillment_type (fulfillment_type_id)
);

CREATE TABLE fulfillment_type
(
    fulfillment_type_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name varchar(50) NOT NULL,

    CONSTRAINT uk_fulfillment_type_name UNIQUE (name),
    CONSTRAINT chk_fulfillment_type CHECK ( name IN ('PICK_UP', 'SHIPPING'))
);
```

Während die Entität Buch (book) die Metadaten des Buches verwaltet, speichert die Tabelle Buchexemplar (book_copy) die konkreten Exemplare eines Buches. Über Fremdschlüssel werden das zugehörige Buch und der Inhaber verknüpft. Es wird explizit auf ein UNIQUE-Constraint verzichtet, da ein Benutzer das gleiche Buch auch mehrfach anbieten könnte.

Die Attribute Status (status) und Leihdauer (loan_duration_days) werden mit NOT NULL gekennzeichnet, da beide für den Ausleihprozess relevant sind. Anhand der Dauer kann das Rückgabedatum berechnet werden, während durch den Status Exemplare temporär (BLOCKED) oder dauerhaft (INACTIVE) von der Leihe ausgenommen werden können.

Die referenzielle Aktion ON DELETE CASCADE auf owner_id ermöglicht es, Buchexemplare zu entfernen, sobald die Person gelöscht wird. Da die Exemplare jedoch auch in den Ausleihvorgängen referenziert werden, wird im Fall einer Referenz das Löschen der Person verhindert und die Person in den Status *INACTIVE* versetzt. So kann die Historie beibehalten werden.

Da ein Buch auf mit unterschiedlichen Bereitstellungsarten angeboten werden kann, ist eine Mehrfachzuordnung von Bereitstellungsarten möglich. Die Zuordnungstabelle *book_copy_fulfillment* löst diese Mehrfachbeziehung auf und ordnet Exemplaren verschiedene Bereitstellungsarten zu. Auch hier greift ein ON DELETE CASCADE, sodass die Zuordnungen entfernt wird, sobald ein Exemplar gelöscht wird.

Die verknüpfte Tabelle *fulfillment_type* enthält ausschließlich einen künstlichen Primärschlüssel sowie eine eindeutige Bezeichnung (name). Durch ein Prüfkriterium wird sichergestellt, dass nur die Werte ‚PICK_UP‘ und ‚SHIPPING‘ zulässig sind.

Buchexemplar

Testprozedur und Ergebnis

```
CREATE OR REPLACE PROCEDURE createBookCopy(p_isbn varchar(13), p_user_id BIGINT, p_loan_duration smallint,
                                           p_condition varchar(50), OUT p_copy_id BIGINT)
--
IF p_book_id IS NULL THEN
    RAISE EXCEPTION 'Copy cannot be created as the book title does not exist. Please create book first';
END IF;

INSERT INTO book_copv (book_id, owner_id, loan_duration_days, condition)
```

```
CREATE OR REPLACE PROCEDURE updateBookCopy(p_book_copy_id BIGINT, p_user_id BIGINT, p_loan_duration smallint,
                                           p_condition varchar(50))
LANGUAGE plpgsql AS
$$
BEGIN
    IF isActionAllowed(p_book_copy_id p_book_copy_id, p_user_id p_user_id) THEN
        UPDATE book_copy
        SET loan_duration_days = p_loan_duration,
            condition = p_condition
        WHERE book_copy_id = p_book_copy_id;
        RAISE NOTICE 'Book copy was updated successfully';
    END IF;
END;
$;
```

```
[2026-08-30 11:38:53] iuLendMyBook> CALL createBookCopy('9781000002003', getuserbyemail('clara.neumann@example.org'),
                                                    10::smallint, 'Guter Zustand', NULL)

Book copy with ISBN 9781000002003 for user 9 successfully created
[2026-08-30 11:38:58] iuLendMyBook.public> CALL updateBookCopy(
    (getBookCopiesByUser('9781000002003', getuserbyemail('clara.neumann@example.org')))[1],
    getuserbyemail('clara.neumann@example.org'), 60::smallint,
    'Zustand OK -> eingerissene Seite 102')

Book copy was updated successfully
[2026-08-30 11:39:01] iuLendMyBook.public> CALL updateBookCopy(
    (getBookCopiesByUser('9781000002003', getuserbyemail('clara.neumann@example.org')))[1],
    getuserbyemail('jonas.reuten@example.org'), 60::smallint,
    'Zustand OK -> eingerissene Seite 102')

[2026-08-30 11:39:01] [P0001] ERROR: Action cannot be performed. The copy may not exist or you are not allowed to perform the selected action
```

Entsprechend der bereits vorgestellten Prozeduren wird mit *createBookCopy* ein neues Exemplar angelegt. Dabei können für eine Person auch mehrere Exemplare des gleichen Buches angelegt werden.

Nach der Erstellung können die Entleihdauer und der Zustand des Buches über eine Update-Prozedur aktualisiert oder das Exemplar über eine Delete-Prozedur gelöscht werden. Die Funktion *isActionAllowed* prüft dabei in beiden Fällen, ob die übergebene User-ID der Owner-ID entspricht oder der übergebene Benutzer Administratorenberechtigungen besitzt. Falls nicht, wird die Änderung oder das Löschen unterbunden.

Die Testfälle zeigen das erfolgreiche Erstellen und Aktualisieren eines Buchexemplars. Der Versuch eine Aktualisierung durch einen User vorzunehmen, der nicht Inhaber des Buchexemplars ist, wird eine entsprechende Ausnahme geworfen.

Buchexemplar

Testprozedur und Ergebnis

```
CREATE OR REPLACE PROCEDURE deleteBookCopy(p_book_copy_id BIGINT, p_user_id BIGINT)
LANGUAGE plpgsql AS
$$
BEGIN
    IF isActionAllowed(p_book_copy_id p_book_copy_id, p_user_id p_user_id) THEN
        DELETE FROM book_copy WHERE book_copy_id = p_book_copy_id;
        RAISE NOTICE 'Book copy was deleted successfully';
    END IF;

EXCEPTION
    WHEN foreign_key_violation THEN
        RAISE NOTICE 'The copy cannot be deleted because it is referenced in at least one loan process. The status will be set to INACTIVE';
        UPDATE book_copy
        SET status_id = (SELECT status_id from status WHERE name = 'INACTIVE')
        WHERE book_copy_id = p_book_copy_id;
END;
$$;
```

```
[2026-08-30 11:39:08] ivLendMyBook.public> CALL deleteBookCopy(
    (getBookCopiesByUser('9781000002003', getuserbyemail('clara.neumann@example.org')))[1],
    getuserbyemail('jonas.reuter@example.org'))
[2026-08-30 11:39:08] [P0001] ERROR: Action cannot be performed. The copy may not exist or you are not allowed to perform the selected action
[2026-08-30 11:39:18] ivLendMyBook.public> CALL deleteBookCopy(
    (getBookCopiesByUser('9781000002003', getuserbyemail('clara.neumann@example.org')))[1],
    getuserbyemail('clara.neumann@example.org'))

Book copy was deleted successfully
[2026-08-30 11:41:58] ivLendMyBook.public> CALL deleteBookCopy(
    (getBookCopiesByUser('9783000001040', getuserbyemail('eva.brandt@example.org')))[1],
    getuserbyemail('eva.brandt@example.org'))

The copy cannot be deleted because it is referenced in at least one loan process. The status will be set to INACTIVE
```

Im Gegensatz zum Erstellen und Aktualisieren eines Exemplars, ist das Löschen nicht ohne Weiteres möglich. Denn durch die Fremdschlüsselbeziehung zu den Ausleihvorgängen (book_loan) müssten auch die verknüpften Objekte mitgelöscht werden. Da die Historie der Ausleihvorgänge jedoch konserviert werden soll, ist das Löschen eines Exemplars nur so lange möglich, wie keine Ausleihvorgänge bestehen. Die Prozedur *deleteBookCopy* fängt daher bei der Ausführung eine mögliche Fremdschlüsselverletzung ab und setzt den Status des Exemplars auf inaktiv, statt es zu löschen.

Die beiden Testfälle zeigen, dass das Löschen möglich ist, wenn keine Ausleihvorgänge bestehen. Sind hingegen Vorgänge vorhanden, wird der Status des Exemplars auf *INACTIVE* gesetzt.

Zeitfenster

Erstellung der Stammdatentabelle *timeslot*

```
CREATE TABLE timeslot
(
    timeslot_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    begin_time   time      NOT NULL,
    end_time     time      NOT NULL,
    day_of_week  smallint NOT NULL,

    CONSTRAINT uk_timeslot UNIQUE (begin_time, end_time, day_of_week),
    CONSTRAINT chk_timeslot_dayOfWeek CHECK ( day_of_week BETWEEN 1 AND 7),
    CONSTRAINT chk_timeslot_endAfterBegin CHECK ( end_time > begin_time)
);
```

Buchexemplare sollen zur Abholung angeboten werden, wobei es dem Anbieter möglich sein soll, Zeitfenster zu definieren in denen eine Abholung möglich ist. Hierfür wird eine Tabelle mit möglichen Zeitfenstern (timeslot) für die Abholung angelegt. Sie enthält die Beginn- (begin_time) und End-Uhrzeit (end_time) des Zeitfensters sowie den zugehörigen Wochentag (day_of_week).

Fachlich wird vorausgesetzt, dass alle drei Attribute für die Erstellung eines Zeitfensters notwendig sind. Daher werden alle Attribute mit Hilfe von NOT NULL als Pflichtfelder gekennzeichnet. Da ein Zeitfenster im Verlauf einer Woche mehrfach vorkommen kann, ist nur die Kombination aller drei Felder eindeutig, sodass das UNIQUE-Constraint auf die Kombination der Attribute gesetzt wird.

Um sicherzustellen, dass bei der Eingabe nur zulässige Zeitfenster entstehen, werden zwei Prüfkriterien festgelegt. So können die Wochentage nur als Zahlenwerte zwischen 1 und 7 angegeben werden. Zudem sollte das Ende eines Zeitfensters nicht vor dem Beginn liegen, sodass dies als weiteres Kriterium ergänzt wird.

Zeitfenster

Testprozedur und Ergebnis

```
[2026-08-26 19:41:58] iuLendMyBook.public> INSERT INTO timeslot (begin_time, end_time, day_of_week)
VALUES ('10:00:00', '12:00:00', 5)
[2026-08-26 19:41:58] 1 row affected in 5 ms
[2026-08-26 19:42:01] iuLendMyBook.public> INSERT INTO timeslot (begin_time, end_time, day_of_week)
VALUES ('10:00:00', '09:00:00', 2)
[2026-08-26 19:42:01] [23514] ERROR: new row for relation "timeslot" violates check constraint
"chk_timeslot_endaftebegin"
[2026-08-26 19:42:01] Detail: Failing row contains (37, 10:00:00, 09:00:00, 2).
[2026-08-26 19:42:03] iuLendMyBook.public> INSERT INTO timeslot (begin_time, end_time, day_of_week)
VALUES ('10:00:00', '09:00:00', 10)
[2026-08-26 19:42:03] [23514] ERROR: new row for relation "timeslot" violates check constraint
"chk_timeslot_dayofweek"
[2026-08-26 19:42:03] Detail: Failing row contains (38, 10:00:00, 09:00:00, 10).
[2026-08-26 19:42:06] iuLendMyBook.public> DELETE FROM timeslot ts WHERE timeslot_id = 11
[2026-08-26 19:42:06] 1 row affected in 6 ms
```

Die Tests zur Bearbeitung der Zeitfenster sind ohne Prozeduren gestaltet, da die Zeitfenster zunächst nur beim Start angelegt werden. Dennoch sollen sie zeigen, dass die Bedingungen der Tabelle greifen.

So kann ein neuer Datensatz angelegt werden, für den die Kombination aus Beginn, Ende und Wochentag noch nicht besteht.

Im Gegenzug schlägt die Anlage fehl, wenn die Prüfkriterien (Ende vor Beginn oder falscher Wochentag) nicht erfüllt sind.

Das im Test verwendete Zeitfenster kann gelöscht werden, da es nicht durch eine Abholoption referenziert wird. Bei bestehender Referenz verhindert die Fremdschlüsselbeziehung die Löschung. Auf ON DELETE SET NULL wird bewusst verzichtet, da dadurch bei bereits vorhandener Abholoption ohne Zeitfenster eine Verletzung des UNIQUE NULLS NOT DISTINCT-Constraints entstehen könnte.

Abholoption

Erstellung der Zuordnungstabelle *pickup_option*

```
CREATE TABLE pickup_option
(
    pickup_option_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_address_id  bigint NOT NULL,
    timeslot_id      bigint,

    CONSTRAINT uk_pickup_option UNIQUE NULLS NOT DISTINCT (user_address_id, timeslot_id),
    CONSTRAINT fk_pickup_option_timeslot FOREIGN KEY (timeslot_id) REFERENCES timeslot (timeslot_id),
    CONSTRAINT fk_pickup_option_user_address FOREIGN KEY (user_address_id) REFERENCES
        user_address (user_address_id)
        ON DELETE CASCADE
);
```

Während die möglichen Zeitfenster (timeslot) zentral definiert werden können, kann jeder Verleiher festlegen, zu welchen dieser Zeitfenster Abholungen an einer Abholadresse möglich sind. Die Zuordnungstabelle Abholoptionen (pickup_option) verknüpft über Fremdschlüsselbeziehungen Abholadressen (user_address) und Zeitfenster (timeslot).

Da eine Kombination aus Abholadresse und Zeitfenster nicht mehrfach auftreten können, wird für beide gemeinsam ein UNIQUE-Constraint gesetzt. Da das Zeitfenster kein Pflichtfeld ist, wird durch den Zusatz NULLS NOT DISTINCT sichergestellt, dass es je Benutzeradresse nur einmal die Angabe *kein Zeitfenster* geben kann.

Aufgrund der Pflichtangabe einer Benutzeradresse innerhalb der Abholoption ist es sinnvoll, die Abholoption zu entfernen, sobald die Benutzeradresse gelöscht wird. Dies wird durch die Aktion ON DELETE CASCADE erreicht.

Abholoption

Testprozedur und Ergebnis

```
[2026-08-26 19:53:45] iuLendMyBook.public> INSERT INTO pickup_option (user_address_id, timeslot_id)
VALUES (3, 12)
[2026-08-26 19:53:45] 1 row affected in 6 ms
[2026-08-26 19:54:13] iuLendMyBook.public> INSERT INTO pickup_option (user_address_id, timeslot_id)
VALUES (8, NULL)
[2026-08-26 19:54:13] 1 row affected in 4 ms
[2026-08-26 20:04:09] iuLendMyBook.public> INSERT INTO pickup_option (user_address_id, timeslot_id)
VALUES (8, NULL)
[2026-08-26 20:04:09] [23505] ERROR: duplicate key value violates unique constraint "uk_pickup_option"
[2026-08-26 20:04:09] Detail: Key (user_address_id, timeslot_id)=(8, null) already exists.
[2026-08-26 19:54:59] iuLendMyBook.public> DELETE from pickup_option WHERE pickup_option_id = 1
[2026-08-26 19:54:59] 1 row affected in 4 ms
[2026-08-26 19:55:50] iuLendMyBook.public> DELETE from pickup_option WHERE pickup_option_id = 7
[2026-08-26 19:55:50] [23503] ERROR: update or delete on table "pickup_option" violates foreign key
constraint "fk_book_loan_pickup_option" on table "book_loan"
[2026-08-26 19:55:50] Detail: Key (pickup_option_id)=(7) is still referenced from table "book_loan".
```

Ähnlich zu den Zeitfenster lassen sich die Funktionen der Abholoptionen mit einfachen SQL-Befehlen testen. So zeigen die Testfälle, dass das Einfügen neuer Einträge mit oder ohne Zeitfensterreferenz möglich ist, sofern diese nicht bereits vorhanden sind.

Gleichsam lassen sich Datensätze aus der Tabelle löschen, sofern sie nicht bereits durch einen Ausleihvorgang referenziert werden. In diesem Fall wird das Löschen aufgrund der Verletzung der Fremdschlüsselbeziehung unterbunden.

Die Anwendung einer referenziellen Aktion ist hier nicht sinnvoll, da die Historie der Ausleihvorgänge bestehen bleiben soll. Die Referenz auf NULL zu setzen oder gar den Ausleihvorgang zu löschen würde diesem Grundsatz widersprechen.

Ausleihvorgang

Erstellung der Tabelle *book_loan*

```
CREATE TABLE book_loan
(
  loan_id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  book_copy_id     bigint      NOT NULL,
  borrower_id      bigint      NOT NULL,
  loan_date        date        NOT NULL,
  return_date      date,
  status           varchar(10) NOT NULL DEFAULT 'REQUESTED',
  fulfillment_type_id bigint      NOT NULL,
  user_address_id  bigint,
  pickup_option_id bigint,

  CONSTRAINT fk_book_loan_copy FOREIGN KEY (book_copy_id) REFERENCES book_copy (book_copy_id),
  CONSTRAINT fk_book_loan_borrowerid FOREIGN KEY (borrower_id) REFERENCES user_account (user_id),
  CONSTRAINT fk_book_loan_fulfillment FOREIGN KEY (fulfillment_type_id, book_copy_id)
    REFERENCES book_copy_fulfillment (fulfillment_type_id, book_copy_id),
  CONSTRAINT fk_book_loan_user_address FOREIGN KEY (user_address_id) REFERENCES user_address
    (user_address_id),
  CONSTRAINT fk_book_loan_pickup_option FOREIGN KEY (pickup_option_id) REFERENCES
    pickup_option (pickup_option_id),
  CONSTRAINT chk_book_loan_returnDate CHECK ( return_date >= loan_date),
  CONSTRAINT chk_book_loan_status CHECK ( status IN ('REQUESTED', 'ON_LOAN', 'RETURNED',
    'CANCELED')),
  CONSTRAINT chk_book_loan_return_status CHECK (
    (status = 'RETURNED' AND return_date IS NOT NULL) OR
    (status <> 'RETURNED' AND return_date IS NULL))
);
```

Die Tabelle `book_loan` bildet die konkreten Ausleihvorgänge innerhalb der Buchtausch-App ab. Jeder Ausleihvorgang bezieht sich auf ein bestimmtes Buchexemplar (`book_copy`) und einen Benutzer (`user_account`), der dieses Exemplar ausleiht.

Um sicherzustellen, dass in der Ausleihe nur die Bereitstellungsarten (`fulfillment_type`) gewählt werden, die auch für das jeweilige Exemplar freigegeben sind, wird ein weiterer Fremdschlüssel definiert, der auf die Kombination von Exemplar (`book_copy`) und den Bereitstellungstyp (`fulfillment_type`) in der Zuordnungstabelle `book_copy_fulfillment` verweist.

Der Status eines neu angelegten Ausleihvorgangs wird durch den DEFAULT-Wert zunächst auf `REQUESTED` gesetzt. Mithilfe eines CHECK-Constraints wird sichergestellt, dass nur die vorgesehenen Status `REQUESTED`, `ON_LOAN`, `RETURNED` und `CANCELED` gespeichert werden können. Ein weiteres CHECK-Constraint verhindert, dass das Rückgabedatum zeitlich vor dem Ausleihdatum liegen darf.

Ausleihvorgang

Testprozedur und Ergebnis

```
CREATE OR REPLACE PROCEDURE createBookLoan(p_copy_id BIGINT, p_borrower_id BIGINT, p_pickup_time time,
                                             p_pickup_day INTEGER, OUT p_loan_id BIGINT)
LANGUAGE plpgsql AS
$$
BEGIN
    IF p_copy_id IS NULL THEN
        RAISE EXCEPTION 'The selected book copy does not exist';
    END IF;
    IF p_pickup_time IS NOT NULL AND p_pickup_day IS NOT NULL THEN
```

```
CREATE OR REPLACE PROCEDURE returnBook(p_isbn varchar(13), p_borrower_id BIGINT, p_return_date date)
LANGUAGE plpgsql AS
$$
BEGIN
    UPDATE book_loan bl
    SET return_date = p_return_date,
        status = 'RETURNED'
    WHERE bl.id = v_loan_id;
    RAISE NOTICE 'Book loan with ID % was successfully updated', v_loan_id;
END IF;
END;
$;
```

```
[2026-08-26 20:47:24] iuLendMyBook.public> CALL createBookLoan((getbookcopiesbyisbn('9783000001031'))[1], getUserByEmail('clara.neumann@example.org'),
NULL, NULL, NULL)
Book loan with ID 34 was successfully created
[2026-08-26 20:47:24] iuLendMyBook.public> CALL createBookLoan((getbookcopiesbyisbn('9783000001031'))[1], getUserByEmail('jonas.reuter@example.org'),
NULL, NULL, NULL)
[2026-08-26 20:47:24] [P0001] ERROR: The book cannot be borrowed because it is not available
[2026-08-26 20:47:31] iuLendMyBook.public> CALL returnBook('9783000001031', getUserByEmail('clara.neumann@example.org'), CURRENT_DATE)
Book loan with ID 34 was successfully updated
[2026-08-26 20:47:51] iuLendMyBook.public> CALL createBookLoan((getbookcopiesbyisbn('9788400003012'))[1], getUserByEmail('jonas.reuter@example.org'),
'10:00:00', 4::smallint, NULL)
[2026-08-26 20:47:51] [P0001] ERROR: The chosen timeslot is not available
[2026-08-26 20:51:18] iuLendMyBook.public> CALL createBookLoan((getbookcopiesbyisbn('9788400003012'))[1], getUserByEmail('jonas.reuter@example.org'),
'09:45:00', 1::smallint, NULL)
Book loan with ID 35 was successfully created
[2026-08-26 20:51:21] iuLendMyBook.public> CALL returnBook('9788400003012', getUserByEmail('jonas.reuter@example.org'),
'2026-01-01')
```

Im Kontext der Ausleihvorgänge wurde die Prozedur *createBookLoan* erstellt. Diese erzeugt anhand des Exemplars, des Entleihers und der Bereitstellungsart einen neuen Ausleihvorgang. Dabei wird überprüft, ob für die jeweils gewählte Bereitstellungsart entweder eine Versandadresse oder eine gültige Abholoption verfügbar sind.

Die Abfrage nutzt die Funktion *isBorrowable*, um zu prüfen, dass sich das Exemplar nicht im 'gesperrt'-Status befindet und es aktuell keinen Eintrag in den Ausleihvorgängen gibt, der den Status *ON_LOAN* oder *REQUESTED* aufweist.

Wird der Ausleihvorgang mit Hilfe der Prozedur *returnBook* beendet, wird automatisch der Status auf *RETURNED* gesetzt und das übergebene Rückgabedatum gesetzt.

Dabei werden auch die Prüfkriterien berücksichtigt, sodass ein Rückgabedatum vor dem Ausleihdatum oder ein falscher Status zur Auslösung eines Fehlers führen.

Ausleihvorgang

Testprozedur und Ergebnis

```
SELECT formatisbn( p_isbn b.isbn) isbn, b.title, b.description, bc.condition, ft.name,
       concat(u.last_name, ', ', u.first_name) owner,
       getdistancekm( p_current_latitude 53.157, p_current_longitude 9.993682,
                    p_latitude a.latitude, p_longitude a.longitude) km
FROM book_copy bc
     INNER JOIN book b 1..n<->1: on bc.book_id = b.book_id
     INNER JOIN language l 1..n<->1: on b.language_id = l.language_id
     INNER JOIN book_copy_fulfillment bcf 1<->1..n: on bc.book_copy_id = bcf.book_copy_id
     INNER JOIN fulfillment_type ft 1..n<->1: on bcf.fulfillment_type_id = ft.fulfillment_type_id
     INNER JOIN user_account u 1..n<->1: on bc.owner_id = u.user_id
     LEFT JOIN user_address ua on bc.owner_id = ua.user_id
     LEFT JOIN address a 1..n<->1: on ua.address_id = a.address_id
     INNER JOIN address_type t on ua.address_type_id = t.address_type_id AND t.name = ft.name
WHERE (isborrowable( p_book_copy_id bc.book_copy_id) AND l.name LIKE 'English%') AND
      (ft.name = 'SHIPPING' OR (ft.name = 'PICK_UP' AND getdistancekm( p_current_latitude 53.157,
                                                                    p_current_longitude 9.993682, p_latitude a.latitude,
                                                                    p_longitude a.longitude) < 100))
ORDER BY name, b.isbn, owner_id
LIMIT 10;
```

	isbn	title	description	condition	nr	owner	km
1	978-1-00-000204-1	The Crown of Ember Bay	A young sailor must unite rival isla...	Gut	PICK_UP	Schwarz, Milan	79.69
2	978-1-00-000201-0	Beneath the Quiet Bridge	A detective investigates a series of...	Gut	SHIPPING	Sommer, David	346.84
3	978-1-00-000201-0	Beneath the Quiet Bridge	A detective investigates a series of...	Gut	SHIPPING	Fischer, Helena	376.57
4	978-1-00-000204-1	The Crown of Ember Bay	A young sailor must unite rival isla...	Gut	SHIPPING	Neumann, Clara	258.96
5	978-1-00-000204-1	The Crown of Ember Bay	A young sailor must unite rival isla...	Gut	SHIPPING	Schwarz, Milan	79.69

Neben den Testprozeduren sind für die Ausleihe vor allem aber die Suchanfragen relevant, da hier nur die Buchexemplare angeboten werden dürfen, die zum einen verfügbar sind und zum anderen den Suchkriterien des Nutzers entsprechen.

Die nebenstehende Suchanfrage simuliert eine komplexe Benutzersuche, bei der die Buchexemplare wiedergegeben werden sollen, die aktuell verfügbar sind, die in englischer Sprache verfasst sind und sich zur Abholung in einem Umkreis von 100 km befinden oder über die Bereitstellungsart ‚Versand‘ zur Verfügung stehen.

Hierbei kommt erneut die Funktion *isBorrowable* zum Einsatz, um ausleihbare Exemplare zu ermitteln. Die Funktion *getDistanceKm* berechnet zudem die Entfernung zwischen dem aktuellen Ort und dem Abholort mit Hilfe der Haversine-Formel

Bewertung

Erstellung der Tabelle *loan_rating*

```
CREATE TABLE loan_rating
(
    rating_id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    loan_id        bigint NOT NULL,
    rating_score    smallint NOT NULL,
    comment         text,

    CONSTRAINT uk_loan_rating_loan UNIQUE (loan_id),
    CONSTRAINT fk_loan_rating_loan FOREIGN KEY (loan_id) REFERENCES book_loan (loan_id)
        ON DELETE CASCADE,
    CONSTRAINT chk_loan_rating_score CHECK ( rating_score BETWEEN 1 AND 5)
);
```

Nach Abschluss eines Ausleihvorgangs kann dieser durch den ausleihenden Benutzer bewertet werden. Zu diesem Zweck wird die Tabelle `loan_rating` angelegt. Sie enthält neben einem künstlichen Primärschlüssel Fremdschlüsselbeziehung zum bewerteten Ausleihvorgang (`book_loan`), eine numerische Bewertung sowie einen optionalen Kommentar.

Da jeder Ausleihvorgang höchstens einmal bewertet werden soll, wird das Fremdschlüsselattribut `loan_id` mit einem `UNIQUE`-Constraint versehen. So wird verhindert, dass mehrere Bewertungen für denselben Ausleihvorgang angelegt werden. Die eigentliche Bewertung ist eine Pflichtangabe und wird durch ein `CHECK`-Constraint auf Werte zwischen eins und fünf begrenzt. Im Gegensatz dazu ist der Kommentar optional und kann für eine ausführlichere Begründung verwendet werden.

Da eine Bewertung unmittelbar von dem zugehörigen Ausleihvorgang abhängig ist und ein Fortbestehen ohne diesen fachlich nicht sinnvoll ist, wird über die referenzielle Aktion `ON DELETE CASCADE` sichergestellt, dass verwaiste Einträge in `loan_rating` entfernt werden, sobald der Ausleihvorgang gelöscht wird.

Bewertung

Testprozedur und Ergebnis

```
CREATE OR REPLACE PROCEDURE createBookRating(p_loan_id BIGINT, p_score INTEGER, p_comment TEXT, OUT
p_rating_id BIGINT)
LANGUAGE plpgsql
AS
$$
DECLARE
    v_isReturned BOOLEAN;
BEGIN
    v_isReturned := EXISTS (SELECT from book_loan bl WHERE loan_id = p_loan_id AND status = 'RETURNED');

    IF(v_isReturned = FALSE) THEN
        RAISE EXCEPTION 'Loan process cannot be rated as it is not completed yet';
    ELSE
        INSERT INTO loan_rating (loan_id, rating_score, comment)
        VALUES (loan_id p_loan_id, rating_score p_score, comment p_comment)
        RETURNING rating_id INTO p_rating_id;

        RAISE NOTICE 'Rating with ID % successfully created', p_rating_id;
    END IF;
END
$;
```

```
[2026-08-30 10:47:11] iuLendMyBook.public> CALL createBookRating(25, 5, 'Fairly easy process; Quick delivery', NULL)
Rating with ID 31 successfully created
[2026-08-30 10:47:56] iuLendMyBook.public> CALL createBookRating(35, 4, 'Quiet okay, nothing to moan about', NULL)
[2026-08-30 10:47:56] [P0001] ERROR: Loan process cannot be rated as it is not completed yet
[2026-08-30 10:48:02] iuLendMyBook.public> CALL createBookRating(6, 7, 'Best book lendign process I've ever experienced', NULL)
[2026-08-30 10:48:02] [23514] ERROR: new row for relation "loan_rating" violates check constraint "chk_loan_rating_score"
[2026-08-30 10:48:02] Detail: Failing row contains (32, 6, 7, Best book lendign process I've ever experienced).
[2026-08-30 10:48:06] iuLendMyBook.public> CALL createBookRating(4, 2, 'Not as good as I expected', NULL)
[2026-08-30 10:48:06] [23505] ERROR: duplicate key value violates unique constraint "uk_loan_rating_loan"
[2026-08-30 10:48:06] Detail: Key (loan_id)=(4) already exists.
```

Das Erstellen einer Bewertung ist von verschiedenen Faktoren abhängig. Während das Erstellen mehrerer Bewertungen für einen Vorgang bereits durch das UNIQUE-Constraint sichergestellt wird, muss die Geschäftslogik, dass eine Abgabe nur möglich ist, wenn der Vorgang beendet ist über eine Prozedur abgebildet werden. *CreateBookRating* prüft dafür zunächst ab, ob der übergebene Ausleihprozess im Status *RETURNED* ist. Falls nicht, wird eine Ausnahme zurückgegeben, andernfalls wird die Bewertung angelegt.

Die Testfälle zeigen, dass bei einem Vorgang, der bereits abgeschlossen ist, die Anlage ohne Weiteres erfolgreich verläuft. Für einen Vorgang, der noch nicht beendet ist, wird hingegen die Fehlermeldung ausgegeben.

Aufgrund des Prüfkriteriums zum Wert der Bewertung (rating_score) schlägt das Einfügen hingegen fehl, wenn der Wert außerhalb des definierten Bereichs liegt. Zudem kann aufgrund des UNIQUE-Constraints keine Bewertung abgegeben werden, sofern bereits ein Datensatz vorliegt.

Bewertung

Testprozedur und Ergebnis

```
CREATE OR REPLACE FUNCTION analyzeRatingsAnonymized(p_user_id BIGINT)
  RETURNS TABLE (isbn varchar(20),title varchar(50),no_of_ratings BIGINT, average_score NUMERIC (6,2))
  LANGUAGE plpgsql
AS
$$
DECLARE
  v_isModerator BOOLEAN;
BEGIN
  v_isModerator := EXISTS (SELECT from user_role ur
    INNER JOIN role r 1..n<->1: on ur.role_id = r.role_id WHERE r.name = 'MODERATOR' AND
      ur.user_id = p_user_id);
  IF (v_isModerator) THEN
    RETURN QUERY (SELECT formatisbn( p_isbn b.isbn), b.title, count(lr.rating_id), round(avg(rating_score)
      ::numeric,2) FROM loan_rating lr
      INNER JOIN book_loan bl 1<->1: on lr.loan_id = bl.loan_id
      INNER JOIN book_copy bc 1..n<->1: on bl.book_copy_id = bc.book_copy_id
      INNER JOIN book b 1..n<->1: on bc.book_id = b.book_id
      GROUP BY b.isbn, b.title
      ORDER BY avg(rating_score));
  ELSE
    RAISE EXCEPTION 'You do not have the permission to analyse rating information';
  END IF;
END
$;
```

	isbn ▼	title ▼	no_of_ratings ▼	average_rating ▼
1	978-3-00-000100-0	Das Echo der stillen Stadt	2	4.5
2	978-3-00-000103-1	Die letzte Fahrt nach Nordhafen	2	4
3	978-1-00-000201-0	Beneath the Quiet Bridge	3	4
4	978-3-00-000104-8	Sommer unter fremden Sternen	14	3.21
5	978-8-70-000403-0	Historien om den skjulte havn	1	3
6	978-8-70-000404-7	Mennesker og digitale vaner	1	3
7	978-1-00-000204-1	The Crown of Ember Bay	3	2.33

Eine zentrale Anforderung aus dem Anforderungskatalog besteht darin, dass es der Rolle Moderator möglich sein muss, die abgegebenen Bewertungen systematisch und anonymisiert auszuwerten.

Die Funktion *analyzeRatingsAnonymized* bildet diese Auswertungsmöglichkeit ab. Anhand der übergebenen User-ID wird zunächst die Berechtigung zur Auswertung geprüft. Ist diese vorhanden werden alle abgegebenen Bewertungen ermittelt und über die Ausleihvorgänge, Buchexemplare und Bücher verknüpft, sodass aggregiert nach ISBN und Buchtitel die Anzahl sowie der durchschnittliche Wert der abgegebenen Bewertungen ermittelt wird. Besitzt ein Nutzer nicht die Rolle Moderator wird die Auswertung mit einer Fehlermeldung abgelehnt.

Effizienz und Indexierung

```
CREATE INDEX idx_book_copy_book ON book_copy (book_id);
CREATE INDEX idx_book_copy_owner ON book_copy (owner_id);
CREATE INDEX idx_book_genre_genre ON book_genre (genre_id);
CREATE INDEX idx_book_author_author ON book_author (author_id);
CREATE INDEX idx_user_role_user ON user_role (user_id);
CREATE INDEX idx_book_loan_borrower_date ON book_loan (borrower_id, loan_date DESC);
CREATE INDEX idx_book_copy_fulfillment_copy ON book_copy_fulfillment (book_copy_id, fulfillment_type_id);
CREATE INDEX idx_book_loan_active_copy ON book_loan (book_copy_id) WHERE status IN ('REQUESTED', 'ON_LOAN');
```

PostgreSQL legt für Primärschlüssel und UNIQUE-Constraints bereits automatisch Indizes an, um betreffende Abfragen effizienter zu gestalten. Fremdschlüsselbeziehungen werden hingegen nicht automatisch indiziert. Um u.a. umgekehrte Suchen (z.B. alle Exemplare eines Buchtitels) effizient zu gestalten, werden für zentrale Fremdschlüssel ebenfalls Indizes angelegt.

Im Falle wiederkehrender komplexer Abfragen, wie der Suche nach verfügbaren Exemplaren über die Funktion *isBorrowable* hilft ein partieller Index auf aktuell ausgeliehene Buchexemplare, verfügbare Exemplare effizienter zu finden.